

ReTRACE: MODEL-RESPONSE WATERMARKING FOR DIFFUSION LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Most language-model watermarks verify a keyed pattern in the emitted token sequence. We study a different signal for masked diffusion language models: how the model reconstructs a completed text after keyed parts of its context are hidden. ReTrace compares the log-likelihood of each observed token under two context ablations. A secret orientation bit specifies the desired sign of this contrast, and the detector counts sign agreements. With domain-separated pseudorandom functions, this count has an exact finite-sample $\text{Binomial}(n, \frac{1}{2})$ null for any text generated independently of the key. During blockwise decoding, ReTrace rejection-samples model proposals at reproducible carrier positions and uses an unconditional fallback after a fixed retry budget. This changes the sampling distribution and is not claimed to be distortion-free, but it never inserts a token that the model did not propose. On LLaDA-8B-Instruct, the best evaluated 512-token settings obtain 0.70–0.80 TPR at 1% FPR with GPT-2 perplexity ratios of $0.87 \times$ – $0.74 \times$ relative to matched controls; verification requires 144 masked sequence evaluations. Detection remains below strong token-based baselines and falls from 0.35 to 0.05 after same-model re-denoising of 10% of positions. Finally, a post-hoc substitution ablation explains the design choice: sharper, better-performing denoising conditionals leave fewer plausible alternatives and therefore less room for substitution.

1 INTRODUCTION

Language-model watermarks usually encode a secret pattern in the generated sequence. Red–green methods bias the probability of keyed token subsets (Kirchenbauer et al., 2023); sampling-based methods correlate token choices with a pseudorandom sequence (Kuditipudi et al., 2024); and recent methods for diffusion language models (DLMs) use denoising order or unresolved context as additional embedding channels (Hong & No, 2026). Their implementations differ, but their detectors can evaluate the watermark from token identities and a key without running the generator.

This paper asks whether model inference itself can provide a useful verification signal. A masked DLM is trained to predict missing tokens from arbitrary visible context. The same operation can therefore be applied to a completed text: hide selected context, run the model, and measure how its predictions change. The central question is whether that change can be keyed, reproduced by a verifier, and steered during generation.

ReTrace implements this idea with paired context ablations. For a generated token y_i , the verifier masks two different subsets of the already completed context and computes

$$g_i(y_i) = \log p_\theta(y_i | C_i^1(y)) - \log p_\theta(y_i | C_i^0(y)). \quad (1)$$

The key supplies an independent orientation bit that determines which sign counts as a match. ReTrace counts matches only at positions where the two signs both have appreciable probability. For a text independent of the key, the orientation bits are fair coins, so the conditional null is exactly $\text{Binomial}(n, \frac{1}{2})$. This guarantee does not depend on a calibration corpus or a Gaussian approximation.

Embedding uses the same observable. During the standard blockwise decode, ReTrace draws a token from the model and checks whether its contrast has the requested sign. At carrier positions

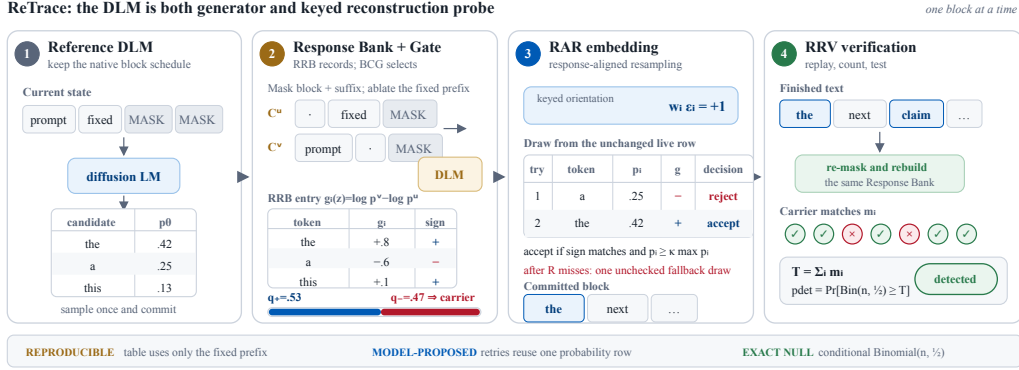


Figure 1: Overall ReTrace pipeline. (1) The reference DLM retains its native block and confidence schedule. (2) The Reproducible Response Bank (RRB) records keyed context-ablation responses, and the Balanced Carrier Gate (BCG) selects positions with substantial probability mass on both signs. (3) At those carriers, the Response-Aligned Resampler (RAR) accepts a plausible proposal with the requested sign, retrying from the unchanged live conditional at most R times before an unconditional fallback. (4) Given the final text, model, and key, the Response-Replay Verifier (RRV) reconstructs the RRB and applies an exact binomial test.

it may draw again from the unchanged conditional, up to a budget R ; after exhaustion it commits one unconditional draw. A key-free plausibility floor rejects guided proposals whose probability is too small relative to the most likely token. The resulting procedure is rejection sampling: it changes token probabilities, but every emitted token is an actual model proposal and retries require no additional model evaluation.

Figure 1 follows a block from the reference decoder through response-bank construction, carrier gating, embedding, and verification. The main contributions are:

- a verifier that derives evidence from a DLM’s response to keyed context ablations, rather than from a key–token statistic alone;
- a reproducible carrier construction and a finite-sample exact binomial test under the ideal-PRF model;
- a response-guided resampling embedder with an explicit retry–quality trade-off; and
- an evaluation that reports detection, matched-control perplexity, generation and verification cost, and post-editing failure modes on LLaDA-8B-Instruct.

At 512 tokens, the strongest evaluated ReTrace settings reach 0.70–0.80 TPR at 1% FPR with measured perplexity ratios of $0.87 \times - 0.74 \times$. These ratios partly reflect the plausibility floor’s preference for high-probability tokens and should not be read as evidence that watermarking improves semantic quality. ReTrace also requires 144 masked sequence evaluations for verification and is substantially less robust to editing than token-based baselines. We present these costs as part of the method’s operating point, not as secondary caveats.

2 RELATED WORK

Diffusion language models. Continuous diffusion (Ho et al., 2020) was extended to discrete data through structured corruption processes (Austin et al., 2021), score-ratio models, and masked-diffusion objectives (Lou et al., 2024; Sahoo et al., 2024; Shi et al., 2024; Ou et al., 2025). More recent systems scale this approach to instruction-following language models, including LLaDA (Nie et al., 2025; Zhu et al., 2025) and Dream (Ye et al., 2025). ReTrace relies on one basic consequence of masked-token training: a completed sequence can be corrupted again and scored under several visible-context patterns.

Practical DLM decoding is neither fully parallel nor order-invariant. Systems commonly complete blocks from left to right and denoise positions within a block (Arriola et al., 2025); the unmasking

order can affect the resulting text (Kim et al., 2025). Work on accelerated decoding likewise finds that committing a token before sufficient context is available can harm coherence (Wu et al., 2026; Wei et al., 2026; Ben-Hamu et al., 2025). ReTrace therefore preserves the model’s reference block and confidence schedule. It changes token selection only through resampling after the model has produced a conditional distribution.

Watermarks for autoregressive models. The dominant family modifies token probabilities. Kirchenbauer et al. (2023) add a logit bias to a keyed green subset of the vocabulary, and detection counts green tokens. Later work studies the reliability, robustness, and statistical guarantees of this design (Kirchenbauer et al., 2024; Zhao et al., 2023; 2024). Another family couples sampling randomness to a keyed pseudorandom sequence, using Gumbel-max sampling (Aaronson & Kirchner, 2023; Fu et al., 2024) or constructions with distortion-freeness and undetectability guarantees (Kuditipudi et al., 2024; Christ et al., 2024). Watermarks can also be placed in model parameters (Block et al., 2025). Surveys describe the broader design space (Petitcolas et al., 1999; Liang et al., 2026), including paraphrase attacks (Krishna et al., 2023) and key extraction (Jovanović et al., 2024). These methods provide an important efficiency reference: their text detectors do not need to execute the generator.

Watermarks beyond left-to-right generation. Autoregressive watermarks cannot always be transferred directly to DLMs because unresolved positions do not have a fixed left-to-right prefix. Chen et al. (2025) avoid prefix dependence when biasing token selection. For DLMs specifically, Gloaguen et al. (2026) apply a red-green bias in expectation over unresolved context, while Bagchi et al. (2025) use keyed Gumbel-max sampling during denoising. Other constructions target order-agnostic or efficient watermarking (Wu et al., 2025; Raban et al., 2026). dgMARK (Hong & No, 2026) instead steers the unmasking order toward positions whose preferred token satisfies a keyed parity condition. We reproduce dgMARK locally because it is a strong DLM-specific baseline with a different embedding channel and a zero-inference detector.

Positioning of ReTrace. ReTrace differs primarily at verification time. Its statistic is not available from token identities and the key alone: the verifier must run the DLM under keyed corruptions and compare the resulting likelihoods. This model-response signal may expose structure that a token hash does not, but it also makes verification substantially more expensive and ties the detector to a particular checkpoint. To our knowledge, the cited language-model watermarks do not use the generator as a reconstruction oracle in this form.

3 BACKGROUND AND PROBLEM SETUP

Blockwise masked diffusion. Let x be a prompt and $y = (y_1, \dots, y_N)$ the generated continuation. We study the semi-autoregressive schedule used by LLaDA: y is partitioned into contiguous blocks, blocks are completed from left to right, and diffusion resolves masks within the active block (Arriola et al., 2025; Nie et al., 2025). When block B_b is active, the prompt and earlier blocks are fixed while B_b and all later blocks remain masked. A masked DLM returns a categorical distribution for every unresolved position in one sequence evaluation; the sampler commits the most confident positions according to the reference schedule.

Verification setting. The embedder receives the model, prompt, secret key K , and message. The verifier later receives only the completed token sequence, the same model, K , and public decoding parameters. It does not receive logits, random seeds, or the order in which tokens were committed. Each document uses a nonce-derived key $K_d = \text{HMAC}(K, \text{nonce}_d)$. The null hypothesis covers any text produced independently of K_d ; it does not require a particular natural-text distribution.

We evaluate three properties. *Validity* requires false-positive control under the null. *Power* is reported as true-positive rate (TPR) at a fixed false-positive rate (FPR). *Cost* includes both changes in text quality and the model evaluations required at generation and verification time. Because ReTrace changes the sampling distribution, quality is always compared with a non-watermarked control using the same base schedule and sampling temperature.

4 RETrace

ReTrace must reproduce at verification time every decision that determines which tokens are scored. This requirement divides the method into four modules. The *Reproducible Response Bank* (RRB) records how the DLM responds to a fixed family of context ablations (§4.1). The *Balanced Carrier Gate* (BCG) admits only positions with usable probability mass on both response signs. The *Response-Aligned Resampler* (RAR) embeds evidence without changing the reference block schedule (§4.3), and the *Response-Replay Verifier* (RRV) reconstructs the RRB from the completed text and applies an exact test (§4.2).

4.1 REPRODUCIBLE RESPONSE BANK

Consider a block B_b beginning at position r_b . Let F_b contain the prompt and all positions before B_b , which are final when the block starts. ReTrace constructs L equal-size ablation patterns $D_{b,1}, \dots, D_{b,L} \subset F_b$. The implementation includes near-context, far-context, and keyed pseudorandom patterns. For pattern ℓ , the model input masks $D_{b,\ell}$, the entire current block, and every later block. For each $i \in B_b$ and vocabulary item v , one masked sequence evaluation yields

$$\ell_{i,\ell}(v) = \log p_\theta(v \mid F_b \setminus D_{b,\ell}; B_b \text{ and suffix masked}). \quad (2)$$

Masking the current block is essential. It prevents tokens committed later in that block from changing the RRB. Masking the suffix is equally important because those tokens were not visible when the embedder built the bank. Given the completed text, the RRV restores exactly this state by re-masking the current block and suffix. Thus the RRB depends on the final prefix, model, public parameters, and key, but not on an unrecorded generation trajectory.

For an unordered pair of patterns (u, v) , define

$$g_i^{u,v}(z) = \ell_{i,v}(z) - \ell_{i,u}(z). \quad (3)$$

Let p_i^{base} denote the model distribution when B_b and the suffix are masked but no context pattern is ablated. The probability mass on each sign is

$$q_{i,+}^{u,v} = \sum_z p_i^{\text{base}}(z) \mathbf{1}[g_i^{u,v}(z) > 0], \quad q_{i,-}^{u,v} = \sum_z p_i^{\text{base}}(z) \mathbf{1}[g_i^{u,v}(z) < 0], \quad (4)$$

and the two-sided score is

$$S_i^{u,v} = 2 \min(q_{i,+}^{u,v}, q_{i,-}^{u,v}). \quad (5)$$

ReTrace chooses, independently at each position, the pair with the largest $S_i^{u,v}$ and stores its contrast as g_i in the RRB. The *Balanced Carrier Gate* then admits position i when $S_i > s_{\min}$; an optional per-block cap retains the largest scores. Because $S_i^{u,v}$ is unchanged when u and v are swapped, the BCG neither observes nor favors the requested orientation. Both pair selection and gating are recomputable from the block-masked state.

4.2 RESPONSE-REPLAY VERIFIER AND EXACT NULL

The key derives an orientation $\varepsilon_i \in \{-1, +1\}$ for every position using a PRF domain separate from pattern generation, pair selection, carrier assignment, and payload grouping. A target $w_i \in \{-1, +1\}$ represents the desired provenance or message bit. For a nonzero contrast, position i matches when

$$m_i = \mathbf{1}[w_i \varepsilon_i g_i(y_i) > 0]. \quad (6)$$

If $g_i(y_i) = 0$ exactly, ReTrace uses an additional domain-separated fair tie bit for m_i . Ties cannot be steered during embedding, but assigning all of them zero would bias the null.

For provenance-only detection, all carriers form a presence pool P with $w_i = +1$. If a payload is enabled, the implementation can reserve a synchronization subset for presence and partition the remaining carriers by payload bit. These groups must remain separate: aggregating a balanced message would cancel positive and negative targets. The presence statistic and its one-sided p -value are

$$T = \sum_{i \in P} m_i, \quad p_{\text{det}} = \Pr[\text{Binomial}(|P|, \frac{1}{2}) \geq T]. \quad (7)$$

Proposition 1 (Conditional exact null). *Assume the PRF domains above behave as independent random functions. Fix any text y generated independently of the document key, and condition on the carrier set, selected pairs, contrast values, payload roles, and target bits. The orientation bits (and tie bits where needed) remain independent fair coins. Consequently, the m_i are independent Bernoulli($\frac{1}{2}$) variables and $T \sim \text{Binomial}(|P|, \frac{1}{2})$ exactly.*

The proposition is conditional on the observed text and therefore requires neither a text distribution nor an asymptotic approximation. Its guarantee does rely on correct domain separation and on the text being independent of the document key under the null. The implementation enforces this separation with distinct HMAC labels.

4.3 RESPONSE-ALIGNED RESAMPLING

Algorithm 1 ReTrace embedding and verification. BUILD-RRB is shared verbatim by the RAR and RRV. The reference block order, confidence selection, and live DLM conditional are unchanged; guided retries reuse the current probability row.

Input: prompt $x_{<B_1}$, key K , retry budget R , floor κ , threshold $s_{\min}$ **Output:** generated text y or $p_{\det}$

BUILD-RRB(x, b, K) — Reproducible Response Bank

- 1 Mask the current block B_b and suffix; construct L keyed ablations of the fixed prefix.
- 2 Evaluate the L ablated sequences and one unablated sequence with the DLM.
- 3 For every $i \in B_b$, select $(u, v) = \arg \max_{u < v} S_i^{u,v}$ and store $g_i = \ell_{i,v} - \ell_{i,u}$ and $S_i = 2 \min(q_{i,+}, q_{i,-})$.

EMBED(x, K, R, κ) — Response-Aligned Resampler

- 4 **for** each block B_b in reference order **do**; compute $\mathcal{R}_b \leftarrow \text{BUILD-RRB}(x, b, K)$.
- 5 **while** B_b contains masks **do**; let the reference confidence rule select positions and obtain each live row p_i .
- 6 **if** $S_i \leq s_{\min}$, draw $z \sim p_i$ once.
- 7 **else** draw at most R proposals from p_i ; accept the first z satisfying $w_i \varepsilon_i g_i(z) > 0$ and $p_i(z) \geq \kappa \max_v p_i(v)$.
- 8 **if** no guided proposal is accepted, draw one fresh unchecked $z \sim p_i$; commit z .

VERIFY(y, K) — Response-Replay Verifier

- 9 Re-mask each completed block and suffix; rebuild $\mathcal{R}_b$ with BUILD-RRB(y, b, K).
- 10 Apply the BCG, compute m_i at admitted carriers, and return the exact binomial tail in Eq. (7).

Algorithm 1 makes the module boundaries explicit. Within each block, the RAR uses the reference confidence order and live conditional p_i . Positions rejected by the BCG are sampled normally. At an admitted carrier, define the acceptance set

$$A_i = \left\{ z : w_i \varepsilon_i g_i(z) > 0 \text{ and } p_i(z) \geq \kappa \max_v p_i(v) \right\}. \quad (8)$$

The RAR draws independently from p_i up to R times and commits the first proposal in A_i . If none is accepted, it makes one fresh unconditional draw from p_i and commits it without inspection. The probability row does not change during these retries, so they do not require additional model evaluations.

This procedure changes the sampling distribution. Let $a_i = p_i(A_i)$ be the acceptance mass and let

$$q_i = \sum_z p_i(z) m_i(z) \quad (9)$$

where $m_i(z)$ applies Eq. (6) and the tie rule to candidate z . Thus q_i is the probability that an unconditional draw is scored as a detector hit, irrespective of the plausibility floor. The committed-token distribution is

$$(1 - (1 - a_i)^R) p_i(\cdot | A_i) + (1 - a_i)^R p_i, \quad (10)$$

and its detector-hit probability is

$$\Pr[m_i = 1] = 1 - (1 - a_i)^R (1 - q_i). \quad (11)$$

When $\kappa = 0$ and the conditional assigns no mass to exact ties, $a_i = q_i$ and Eq. (11) reduces to $1 - (1 - q_i)^{R+1}$. Separating a_i and q_i is necessary when the plausibility floor or tie rule is active: a fallback token may score as a hit even if it could not have been accepted as a guided proposal.

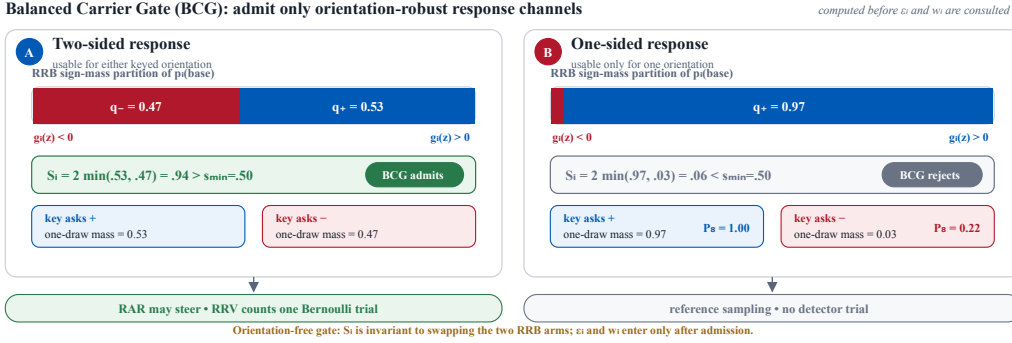


Figure 2: The Balanced Carrier Gate separates usable response channels from orientation-dependent ones. The BCG admits a position only when both signs carry substantial mass under the RRB’s block-masked conditional. Its score is invariant to swapping the response arms and is computed without consulting the keyed orientation. Admitted positions are steered by the RAR and counted by the RRV; rejected positions follow the reference sampler and contribute no detector trial.

The floor bounds the model NLL of every *guided accepted* proposal to within $-\log \kappa$ of the live argmax. It does not guarantee better semantic quality, and the unconditional fallback remains unrestricted. Similarly, saying that every output was proposed by the model is a support statement, not a distortion-free claim. Equation (10) makes the distributional change explicit.

The score S_i is computed under the reproducible block-masked distribution, whereas generation samples from the evolving live conditional. It is therefore a symmetric proxy for live acceptance mass, not a guarantee that each carrier is easy to steer. Selecting the best pair per position increases the measured mean two-sided score from 0.28 for a fixed near/far pair to 0.45. This gate, the unconditional fallback, and the plausibility floor are the three safeguards that determine ReTrace’s empirical trade-off.

4.4 WHY NOT SUBSTITUTE TOKENS AFTER GENERATION?

An earlier design first generated an unwatermarked text and then replaced selected tokens with alternatives having the desired contrast sign. A candidate z was admissible only if its probability relative to the most likely token satisfied $p(z)/p(z^*) \geq e^{-\tau}$. This approach is simple and uses the same detector, but the experiments show that it has too little capacity on the evaluated model.

Across 36 settings covering three carrier rules, three fluency budgets, and two guidance weights, the median selected position had only one admissible token at $\tau = 3$ ($p(z)/p(z^*) \geq 0.05$). The best setting within a $1.35\times$ perplexity budget reached only 0.10 TPR at 1% FPR, compared with 0.86 for the local dgMARK reproduction at $1.23\times$.

A sampler correction made the limitation clearer. Confidence had initially been computed from Gumbel-perturbed logits rather than the clean model distribution. Fixing this error reduced draft perplexity from 22.2 to 9.5 and mean conditional entropy from 0.655 to 0.319. At the same time, the mean number of admissible alternatives fell from 2.5 to 1.6 at $\tau = 3$ and from 14.1 to 5.6 at $\tau = 6$. In this setting, sharper conditionals improve the base sampler while removing the alternatives needed for post-hoc substitution.

This result motivates resampling rather than a universal claim that substitution must fail for every DLM. ReTrace makes its choice while the model is already sampling and falls back to an unconditional proposal when steering fails; it does not force a replacement after the surrounding text has been fixed.

5 EXPERIMENTS

We evaluate four questions: whether response-guided resampling produces detectable signal; how retry budget and plausibility filtering affect measured quality; how ReTrace compares with token-based baselines at their actual compute budgets; and how quickly the signal degrades after editing.

Method	Setting	PPL ↓	Ratio ↓	TPR at FPR ↑			Gen. eval.	Det. eval.
				5%	1%	0.1%		
Budget: one sequence evaluation per generated token								
dgMARK (Hong & No, 2026)	$k = 1$, 256 tok	15.40	1.55×	0.88	0.74	0.62	256	0
KGW (Kirchenbauer et al., 2023)	$\delta = 1$, 256 tok	11.96	1.03×	0.97	0.93	0.73	256	0
KGW (Kirchenbauer et al., 2023)	$\delta = 3$, 256 tok	14.05	1.21×	1.00	1.00	1.00	256	0
Post-hoc substitution	best under 1.35× PPL	30.5	1.17×	0.10	0.10	0.00	256	9
Budget: 1.28× decode — ReTrace Response Bank plus reference schedule								
ReTrace	$R=1$, 512 tok	11.0	0.97×	0.57	0.43	0.20	656	144
ReTrace	$R=2$, 512 tok, held-out	—	1.14×	0.83	0.73	0.63	656	144
ReTrace	$R=4$, 512 tok, held-out	—	1.40×	0.90	0.83	0.77	656	144
ReTrace	$R=16$, $\kappa=0.05$, 512 tok	—	0.74×	0.90	0.80	0.60	656	144
ReTrace	$R=8$, $\kappa=0.1$, 512 tok	5.67	0.87×	0.80	0.70	0.60	656	144
ReTrace	$R=8$, $\kappa=0.1$, 1024 tok	—	0.83×	0.81	0.75	0.69	1312	288
Budget: ~4× decode (one-step lookahead per candidate)								
dgMARK (Hong & No, 2026)	+3-beam, 256 tok	12.23	1.23×	0.92	0.86	0.82	~1024	0

Table 1: Detection, matched-control perplexity, and compute. Rows are grouped by generation budget and use one machine, checkpoint, and prompt protocol. *Ratio* is PPL relative to a non-watermarked control under the same decoding regime; a ratio below one does not imply better semantic quality. *Eval.* counts sequence evaluations, not batched wall-clock invocations. ReTrace uses 144 extra evaluations to construct Response Banks during a 512-token decode and 144 more for verification. dgMARK three-beam uses approximately four generation evaluations per token. KGW is scored on distinct bigrams; without deduplication, its unwatermarked control has 37% FPR. dgMARK arms use $n = 50$ and KGW arms $n = 30$; principal 512-token ReTrace rows use $n = 30$ ($n = 20$ for $R = 16$), and the 1024-token row uses $n = 16$. PPL ratios use only pairs passing the decoded-length rule.

5.1 EXPERIMENTAL SETUP

All experiments use GSAI-ML/LLaDA-8B-Instruct in fp16 on one TITAN RTX. Prompts come from C4 realnewslike and are truncated to 300 characters, following the dgMARK evaluation protocol. Unless noted otherwise, the model generates 512 tokens in 32-token blocks with the number of denoising steps equal to the generation length. ReTrace uses eight ablation patterns, $s_{\min} = 0.5$, full-vocabulary sampling at temperature 0.8, and a per-document HMAC-derived key. The provenance experiments assign every carrier to the presence pool; they do not evaluate payload recovery.

Quality is GPT-2-large perplexity (PPL). For ReTrace, every watermarked document is paired with a control generated by the same scheduler with steering disabled. We report $\exp(\text{mean}(\text{NLL}_{\text{wm}} - \text{NLL}_{\text{ctrl}}))$, so a ratio below one means that the external PPL model preferred the watermarked sample on average. It does not by itself establish better coherence or task performance. Documents shorter than the stated validity threshold after decoding are excluded from paired PPL, and the remaining count is reported. Detection uses the exact binomial tail from Eq. (7); TPR is measured at fixed analytic FPR thresholds.

Compute is reported in *sequence evaluations*. A 512-token reference decode uses 512 evaluations. ReTrace adds nine masked sequences per block—eight ablations and one unablated conditional—for 144 additional evaluations during generation and another 144 at verification. The ablations are batched in the implementation, so sequence evaluations are hardware-independent but are not the same as wall-clock model invocations.

5.2 DETECTION-QUALITY TRADE-OFF

Retries strengthen the signal but can select unlikely tokens. On 30 held-out 512-token prompts, increasing R from 2 to 4 to 8 raises TPR at 1% FPR from 0.73 to 0.83 to 0.87 (Figure 3b). No extra DLM evaluation is needed because all retries reuse the current probability row. The corresponding PPL ratios, however, rise from 1.14× to 1.40× and 1.61×. This trend is consistent with Eq. (10): additional attempts make acceptance more likely, but they also increase the chance of selecting a lower-probability member of the acceptance set.

At $R = 1$, a disjoint 40-document run obtains 0.38 TPR at 1% FPR and a $0.98\times$ paired PPL ratio, although only 17 pairs pass the decoded-length criterion. Length provides another source of power:

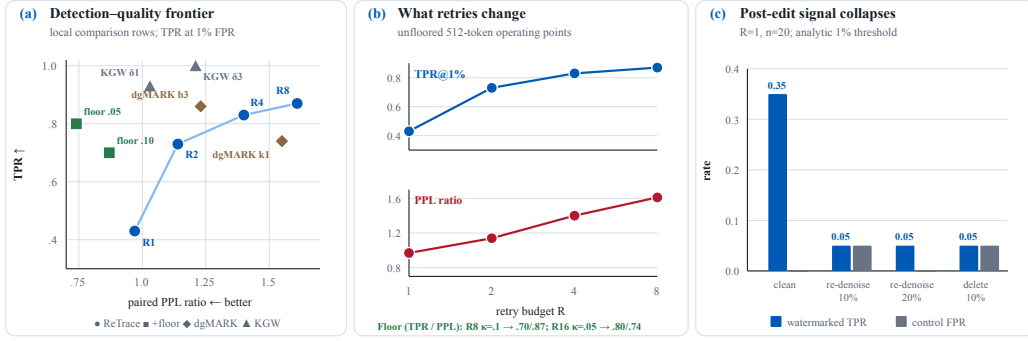


Figure 3: The empirical operating region and its main failure mode. (a) Detection versus matched-control PPL for the local comparison rows; ReTrace uses 512-token outputs and token-based baselines use 256, so this is an operating-point summary rather than a length-controlled ranking. Marker shape separates ReTrace, its plausibility-floor settings, dgMARK, and KGW. (b) For unfloored ReTrace, retries increase both TPR and PPL ratio ($n = 30$ for $R = 2/4/8$; the $R = 1$ point is the table row). (c) On 20 $R = 1$ documents, re-denoising or deleting only 10% of positions reduces TPR from 0.35 to 0.05; the 0.05 resolution reflects the small sample.

at $R = 1$, increasing the continuation from 256 to 512 tokens raises TPR at 1% FPR from 0.25 to 0.40 by increasing the number of carriers.

The plausibility floor controls measured PPL cost. With $R = 8$ and $\kappa = 0.1$, 30 held-out 512-token documents reach 0.70 TPR at 1% FPR and a $0.87 \times$ PPL ratio. The mean sign-match rate falls from 0.711 without the floor to 0.620, while the matched control remains at 0.501. Decoded bigram repetition is 0.33 for ReTrace and 0.36 for its control under the shared validity rule. A smaller floor with more retries ($R = 16$, $\kappa = 0.05$) reaches 0.80 TPR and a $0.74 \times$ ratio on 20 documents, with 15 valid PPL pairs.

Ratios below one are expected to be possible because the floor explicitly favors tokens with high probability under the live DLM conditional, which can also be rewarded by GPT-2-large. We therefore interpret these numbers as “no PPL penalty detected” rather than as evidence that the watermark improves text. At 1024 tokens, $R = 8$, $\kappa = 0.1$ reaches 0.75 TPR at 1% and 0.69 at 0.1%, with a $0.83 \times$ PPL ratio. This longer setting is less conclusive: only 8 of 16 pairs meet the quality-validity rule, and bigram repetition increases from 0.565 in the control to 0.623.

5.3 COMPARISON WITH TOKEN-BASED BASELINES

Table 1 separates methods by generation budget. At one decode, KGW reaches 0.93 TPR at 1% FPR with a $1.03 \times$ ratio, but requires strictly left-to-right DLM generation. The local dgMARK reproduction reaches 0.74 at $1.55 \times$ with one candidate and 0.86 at $1.23 \times$ with three-beam lookahead. The latter uses roughly four sequence evaluations per generated token.

ReTrace’s 512-token decode costs $656/512 = 1.28 \times$ as many sequence evaluations as the reference sampler. Its best measured PPL settings reach 0.70–0.80 TPR at 1% FPR, below both KGW and three-beam dgMARK. ReTrace’s distinguishing result is therefore not state-of-the-art detection. It is the existence of a usable operating point for a model-response statistic with an exact conditional null. The cost is substantial at verification: 144 masked sequence evaluations, compared with zero model evaluations for KGW and dgMARK.

The baseline comparison also has protocol qualifications. KGW is evaluated with distinct bigrams because repeated bigrams share the same green-list event; without deduplication, the unwatermarked left-to-right control measured a 37% false-positive rate. Strict left-to-right LLaDA also has a $1.17 \times$ PPL cost relative to block decoding and high repetition. Conversely, the local dgMARK reproduction is weaker than the result reported in its source paper. The table is a same-machine reproduction, not a claim that these rows are definitive performance ceilings for either baseline.

5.4 DETECTOR CHECKS AND ABLATIONS

The exact-null claim in Proposition 1 is analytical. Current ReTrace control arms provide only a finite-sample sanity check: the 30-document retry experiment and the 40-document $R = 1$ confirmation each produce zero detections at the 1% threshold. These samples are too small to estimate a 1% false-positive rate precisely, and we do not use them as a substitute for the theorem.

On identical cached 512-token outputs, a gated sign count, a gated likelihood-ratio score, and an all-position likelihood-ratio score each reach 0.38 TPR at 1% FPR for $R = 1$. Counting all positions without the two-sided gate falls to 0.33. Since the likelihood scores add complexity without measured gain at this operating point, the reported ReTrace detector uses the gated binomial count.

5.5 POST-EDITING ROBUSTNESS

The response signal is fragile to edits. On 20 $R = 1$ documents, same-model argmax re-denoising of a random 10% of positions reduces TPR at 1% FPR from 0.35 to 0.05; deleting 10% of tokens has the same measured effect. Attacked control arms remain near the nominal threshold. The failure is consistent with the construction: challenges in a later block condition on the completed prefix, so one early edit can alter many downstream contrasts.

This experiment rules out post-edit robustness as a current strength of ReTrace. Token-local schemes retain an important advantage when text is likely to be paraphrased, deleted, or re-denoised. Content-anchored challenges or an edit-synchronization layer may help, but neither is implemented or evaluated here.

5.6 LIMITATIONS

The evaluation uses one DLM checkpoint, one corpus, and small confirmatory samples ($n = 16\text{--}40$ for the principal ReTrace rows). PPL and bigram repetition do not fully measure semantic quality, and the longest generations have substantial validity attrition. ReTrace also requires the verifier to possess the exact model checkpoint and to run it hundreds of times per document. Finally, carrier supply depends on conditional entropy: sharp distributions offer few positions with mass on both response signs. These limitations leave cross-model generality, downstream-task quality, and scalable verification open.

6 CONCLUSION

ReTrace uses a diffusion language model as part of the verifier. It masks a completed text under two keyed context ablations and tests whether the model’s likelihood contrast agrees with secret orientation bits. Domain separation gives the resulting count an exact binomial null, while block masking lets the verifier reconstruct the same challenges used during generation without access to the decoding trajectory.

The experiments establish a trade-off rather than a uniformly better watermark. Guided resampling yields useful detection at low measured perplexity cost, and a plausibility floor prevents large retry budgets from selecting very unlikely proposals. In return, verification requires model inference and the watermark is fragile to post-editing. The evaluation is also limited to one model and modest sample sizes. These limitations make ReTrace most appropriate as a proof of concept for model-response verification, not yet as a replacement for inexpensive token-based detectors. Improving edit robustness and validating the response channel across model families are the main next steps.

REFERENCES

- Scott Aaronson and Hendrik Kirchner. Watermarking gpt outputs, 2023. <https://www.scottaaronson.com/talks/watermark.ppt>.
- Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T. Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025.

- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.
- Arka Bagchi, Akhil Bhimaraju, Moulik Choraria, Daniel Alabi, and Lav R. Varshney. Watermarking discrete diffusion language models. *arXiv preprint arXiv:2511.02083*, 2025.
- Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *NeurIPS*, 2025.
- Adam Block, Alexander Rakhlin, and Ayush Sekhari. Gaussmark: A practical approach for structural watermarking of language models. In *ICML*, 2025.
- Ruibo Chen, Yihan Wu, Yanshuo Chen, Chenxi Liu, Junfeng Guo, and Heng Huang. A watermark for order-agnostic language models. In *ICLR*, 2025.
- Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *COLT*, 2024.
- Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. Gumbelsoft: Diversified language model watermarking via the gumbelmax-trick. In *ACL*, 2024.
- Thibaud Gloaguen, Robin Staab, Nikola Jovanović, and Martin Vechev. Watermarking diffusion language models. In *ICLR*, 2026. arXiv:2509.24368.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Pyo Min Hong and Albert No. dgMARK: Decoding-Guided Watermarking for Diffusion Language Models. In *ICML*, 2026. arXiv:2601.22985.
- Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *ICML*, 2024.
- Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham M. Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *ICML*, 2025.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *ICML*, 2023.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *ICLR*, 2024.
- Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John F. Wieting, and Mohit Iyyer. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. In *NeurIPS*, 2023.
- Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *TMLR*, 2024.
- Yuqing Liang, Jiancheng Xiao, Wensheng Gan, and Philip S. Yu. Watermarking techniques for large language models: A survey. *Artificial Intelligence Review*, 2026.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *ICML*, 2024.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *NeurIPS*, 2025.
- Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *ICLR*, 2025.
- Fabien A. P. Petitcolas, Ross J. Anderson, and Markus G. Kuhn. Information hiding — a survey. In *Proceedings of the IEEE*, 1999.

- Ohad Raban, Ethan Fetaya, and Gal Chechik. LR-DWM: Efficient watermarking for diffusion language models. *arXiv preprint arXiv:2601.12376*, 2026.
- Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Marroquin Marroquin, Alexander M. Rush, Yair Schiff, Justin T. Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.
- Qingyan Wei, Yaojie Zhang, Zhiyuan Liu, Pengfei Zeng, Yu Wang, Biao Qi, Dong Liu, and Linfeng Zhang. Accelerating diffusion large language models with slowfast sampling: The three golden principles. In *ICLR*, 2026.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. In *ICLR*, 2026.
- Linyu Wu, Linhao Zhong, Wenjie Qu, Yifan Li, Yue Liu, Shengfang Zhai, Chunhua Shen, and Jiaheng Zhang. DMark: Order-agnostic watermarking for diffusion large language models. *arXiv preprint arXiv:2510.02902*, 2025.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Xuandong Zhao, Yu-Xiang Wang, and Lei Li. Protecting language generation models via invisible watermarking. In *ICML*, 2023.
- Xuandong Zhao, Prabhajan V. Ananth, Lei Li, and Yu-Xiang Wang. Provable robust watermarking for ai-generated text. In *ICLR*, 2024.
- Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.